

A Flexible End-to-End Radio-Frequency Machine-Learning Toolbox for Rapid Channel-Aware Experimentation

[TODO: Author names and order]

[TODO: Department and institution]

[TODO: City, State, USA]

[TODO: Email addresses]

Abstract—Radio-frequency machine-learning (RFML) experiments often require separate tools for waveform synthesis, channel modeling, dataset preparation, model training, and field-data analysis. Connecting these tools repeatedly increases setup effort and makes it difficult to preserve consistent signal and channel metadata across an experiment. This paper presents a flexible, research-enabling toolbox that integrates parameterized waveform generation, channel augmentation, dataset management, neural-network training, evaluation, and visualization in a unified end-to-end workflow. Its channel layer supports configurable impairments and stochastic models together with reusable interfaces for importing Sionna-compatible digital twins, measured channel responses, and recorded in-phase and quadrature (I/Q) data; any of these can be applied across an entire dataset with per-example parameters drawn from a single reproducible seed. Generated examples retain waveform, channel, and provenance metadata, enabling controlled comparisons among clean, simulated-channel, and measurement-augmented datasets. The principal contribution is the integrated workflow: it allows researchers to move rapidly from a signal or propagation hypothesis to a repeatable training and evaluation experiment without developing a new data pipeline for each study. A two-stage live demonstration is shown to illustrate the toolbox’s adaptability to classifying waveforms in specific realistic scenarios including UT campus and Hat Creek Radio Observatory.

Index Terms—automatic modulation classification, channel augmentation, digital twin, RF machine learning, Sionna RT, spectrum sensing, spectrum sharing

I. INTRODUCTION

Spectrum-sharing systems depend on timely and trustworthy observations of the radio environment. Automatic modulation classification (AMC) can contribute to this situational awareness by assigning received samples to candidate signal families without complete transmitter-side information. Reported classification performance, however, can overstate field performance when the training and test sets share the same signal-to-noise ratio (SNR), channel distribution, or collection geometry. A model trained only on idealized waveforms may learn features that are unstable under location-dependent multipath or receiver impairments [1].

Conducting such studies commonly requires researchers to connect waveform generators, propagation simulators, measurement files, learning frameworks, and visualization utilities through experiment-specific scripts. This fragmentation slows exploratory work and creates opportunities for inconsistent

preprocessing, sample-rate assumptions, data partitions, and metadata. A reusable end-to-end environment can reduce this integration burden while making clean, simulated, and measured conditions easier to compare under a common experimental configuration.

This work presents an interactive radio-frequency machine-learning (RFML) toolbox for that purpose. The software links waveform generation, channel augmentation, visualization, dataset construction, model training, inference, and evaluation through a shared desktop workflow. Each example remains associated with the waveform parameters, channel configuration, source asset, and processing decisions used to produce it. Researchers can therefore change a waveform, channel source, digital twin, or learner without reconstructing the remainder of the experiment. The toolbox is intended to support rapid hypothesis testing while retaining the provenance needed for repeatable comparisons and later field validation.

The work is motivated by the National Science Foundation (NSF) Spectrum Innovation Initiative: National Radio Dynamic Zones (SII-NRDZ) program. An NRDZ is a spatial region, potentially spanning noncontiguous frequencies, in which automated spectrum-management mechanisms control electromagnetic energy entering, leaving, or occupying the zone [2]. The MITRE-led SPARKIE effort is developing an experimental dynamic-spectrum-sharing environment at the Hat Creek Radio Observatory (HCRO), where radio astronomy observations must be protected from harmful interference while permitting controlled spectrum use [3]. Site-specific propagation and repeatable RFML experiments are relevant to this objective, although the classifier described here does not implement a zone-management policy itself.

The paper’s primary contribution is a open sourced, flexible and easy-to-use toolbox for rapid and repeatable RFML experimentation. First, its unified architecture connects MATLAB-based waveform synthesis, configurable impairments, stochastic and deterministic channel modeling, dataset management, PyTorch-based training, inference, and visualization. Second, it provides interchangeable input paths for user-supplied Sionna/Mitsuba digital twins, complex channel responses stored in MATLAB or NumPy files, and recorded I/Q data represented with the Signal Metadata Format (SigMF), while preserving provenance across the workflow. Third, the

paper shows how these components can be combined without rebuilding the surrounding data pipeline, through a live end-to-end demonstration that moves from waveform synthesis and channel augmentation to training and to evaluation on field recordings. The bundled University of Texas at Austin (UT Austin) scene, an independently developed HCRO scene, and channel estimates from the channel-measurement-augmented training (CMAT) pipeline [4] serve as example inputs which can be swapped and extended as preferred by users of the tool kit.

II. BACKGROUND AND MOTIVATION

Deep neural networks can learn modulation-dependent structure directly from time-domain I/Q samples, but their performance depends on the distribution represented during training. Prior over-the-air studies have shown that multipath, carrier-frequency offset, symbol-rate variation, and other acquisition effects can change classification performance materially [1]. These factors are not nuisance variables in a spectrum-sharing setting. A sensing receiver may move, observe an emitter through a previously unseen path, or operate near a protected service whose interference environment changes with location and time.

Channel augmentation provides a structured means of introducing propagation variation before field data are available. Stochastic models can generate many channel realizations efficiently, whereas deterministic ray tracing connects propagation paths to a specific three-dimensional geometry. Sionna RT provides hardware-accelerated computation of path coefficients, delays, angles, and channel responses for configured scenes, antenna arrays, and transmitter–receiver locations [5]. Imported channel responses and field recordings provide complementary, observation-based inputs. None of these paths is tied to one site: scene and channel assets are selected by the operator, passed through common downstream data and learning components, and retained with the generated data. This interchangeability allows a researcher to begin with inexpensive stochastic tests, introduce a site-specific digital twin or measured channel when available, and evaluate the resulting model on field data without rebuilding the experiment. Ray-tracing fidelity nevertheless depends on the geometry, material assignments, antenna models, and solver configuration, while measured inputs depend on their calibration and metadata. Fig. 1 summarizes the resulting data paths.

III. TOOLBOX ARCHITECTURE

The toolbox is implemented as a PySide6 desktop application organized around the experiment lifecycle rather than a single propagation model, waveform family, or classifier. Separate views support waveform generation, channel augmentation, and dataset visualization. Additional views support model training, inference, and evaluation. A shared dataset manager transfers samples and metadata between these views so that an operator can perform a complete experiment without assembling separate conversion and orchestration scripts. MATLAB supplies the waveform-generation routines. NumPy

and SciPy support signal processing, PyTorch and TensorFlow support learning, and Sionna provides ray tracing.

A. Waveform Synthesis and Inspection

The waveform generator accepts the sampling frequency f_s , carrier frequency f_c , symbol period T_s , modulation order M , number of symbols, pulse shape, roll-off factor, and output representation. Thirteen selectable families are implemented: pulse-amplitude modulation (PAM), quadrature-amplitude modulation (QAM), phase-shift keying (PSK), frequency-shift keying (FSK), frequency-hopping spread spectrum (FHSS), linear frequency modulation (LFM), Barker-coded pulses, frequency-modulated continuous wave (FMCW), Wi-Fi, Long Term Evolution (LTE), fifth-generation New Radio (5G NR), IEEE 802.15.4 offset quadrature phase-shift keying (Zigbee), and chirp spread spectrum (LoRa). PAM, QAM, and PSK also return the underlying symbols for constellation inspection. The interface displays time-domain, spectral, constellation, and spectrogram views before a sample is stored.

All thirteen selections call a single MATLAB dispatcher through the MATLAB Engine for Python. The dispatcher returns exactly $N = N_{\text{sym}} \text{round}(f_s T_s)$ samples. PAM, QAM, and PSK use the selected root-raised-cosine (RRC) or rectangular pulse shape. The remaining families bypass that filter and use the constructions in Table I. Native-rate standards waveforms are resampled to f_s where required and then fitted to N : a frame shorter than N is repeated, and a longer one is clipped at a randomly chosen offset so that repeated calls do not return the same deterministic preamble. Each family can be returned as complex baseband or converted to a real passband signal by quadrature upconversion at f_c ; complex baseband is the default, since both the stochastic and ray-traced channel paths require it.

The linear-modulation functions use Communications Toolbox and Signal Processing Toolbox primitives. LFM, Barker, and FMCW use Phased Array System Toolbox; Wi-Fi, LTE, and 5G NR use WLAN, LTE, and 5G Toolbox, respectively; and Zigbee uses the Communications Toolbox `lrwpan` package. LoRa is generated in closed form and requires no toolbox, so it remains available when MATLAB is absent. Both real passband and complex baseband outputs are available. The stochastic and ray-traced channel paths require complex baseband I/Q input; converting a real passband array to a complex data type is not considered an equivalent representation. This constraint is enforced by the channel interface.

B. Channel Augmentation

Four channel paths expose different levels of propagation detail. The first applies additive white Gaussian noise (AWGN), scalar amplitude and phase changes, and carrier-frequency offset. The second applies a stochastic tapped-delay-line channel with a configurable profile, delay spread, SNR, and random seed. The third obtains time-domain channel taps from Sionna RT after loading an operator-selected scene and specifying transmitter and receiver positions, antenna

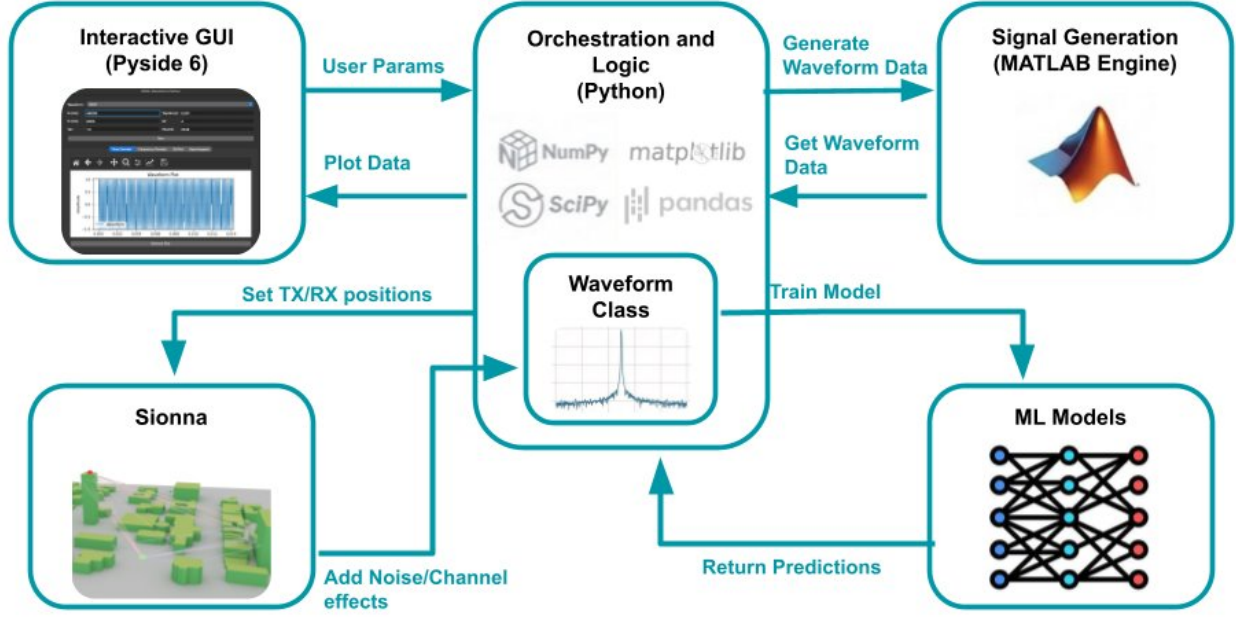


Fig. 1. Toolbox architecture. A unified waveform backbone provides consistent control across the generation, visualization, channel-augmentation, and model-training pipelines. Synthetic and recorded I/Q data share common data and metadata interfaces, and each processing step retains provenance.

TABLE I
WAVEFORM IMPLEMENTATIONS USED BY THE TOOLBOX.

Family	MATLAB primitive	Construction	Parameters used
PAM	pammod, rcosdesign, upfirdn	Unit-power random symbols with RRC or rectangular shaping.	M , roll-off, span, and pulse shape.
QAM	qammod, rcosdesign, upfirdn	Unit-power random symbols with RRC or rectangular shaping.	M , roll-off, span, and pulse shape.
PSK	pskmod, rcosdesign, upfirdn	Gray-coded symbols with RRC or rectangular shaping.	M , roll-off, span, and pulse shape.
FSK	Local continuous-phase generator	One orthogonal tone per symbol with phase continuity.	M ; spacing $1/T_s$.
FHSS	Local continuous-phase generator	One random hop per T_s with phase continuity.	M hops; spacing $f_s/(2M)$.
LFM	phased. LinearFMWaveform	Pulsed linear chirp.	Bandwidth $Mf_s/8$; width up to $20T_s$.
Barker	phased.PhaseCodedWaveform	Barker-coded pulse.	M selects the nearest valid Barker length.
FMCW	phased.FMCWWaveform	Triangular continuous-wave sweep.	Bandwidth $Mf_s/8$; sweep up to $50T_s$.
Wi-Fi	wlanWaveformGenerator	Random payload in a high-efficiency single-user packet, then re-sampled.	Fixed 20-MHz channel; modulation/coding index 0.
LTE	lteRMCDLTool	R.9 downlink reference-measurement waveform; natively 30.72 Msps.	Fixed R.9 profile; 20 MHz, 100 resource blocks.
5G NR	nrWaveformGenerator	Downlink carrier with grid, bandwidth part, and PDSCH allocation widened to fill the carrier; natively 30.72 Msps.	Fixed 20-MHz FR1 profile; 106 resource blocks, 10 subframes.
Zigbee	lrrpanWaveformGenerator	IEEE 802.15.4 O-QPSK PHY with half-sine chip shaping, then resampled.	Fixed 2450-MHz band; 2 Mchip/s.
LoRa	Local chirp-spread-spectrum generator	Preamble, sync word, and quarter-symbol SFD followed by cyclically shifted payload chirps with analytic phase continuity.	M selects the spreading factor (7–12); T_s sets the oversampling factor and hence the chirp bandwidth.

arrays, center frequency, sampling rate, and propagation-solver parameters. The fourth draws from a bank of imported channel responses: files are loaded into a list of impulse responses, and

either a selected entry or a randomly drawn one is applied. Any of the four can be applied to a single dataset for inspection or swept across an entire dataset folder in one pass. In the

latter case each file draws its own parameters, seed, and—for the measured bank—its own channel from a generator seeded once per run, so a sweep varies per example while remaining reproducible from that seed.

For a discrete-time multiple-input multiple-output channel, the received sample at antenna r is represented as

$$y_r[n] = \sum_{t=1}^{N_{tx}} \sum_{\ell=0}^{L-1} h_{r,t}[n, \ell] x_t[n - \ell] + w_r[n], \quad (1)$$

where $h_{r,t}[n, \ell]$ is the channel tap from transmit antenna t to receive antenna r , and $w_r[n]$ is receiver noise. For a static scene and stationary endpoints, the tap sequence is time invariant over one generated example. The current implementation normalizes the input waveform to unit average power, applies the configured transmit power, pads or truncates it to the requested length, evaluates the Sionna time-domain channel, adds noise at the configured level, and returns a selected receive-antenna stream as complex I/Q.

The ray-tracing view reports the loaded scene, computed paths, and channel-tap status and supports visual comparison of clean and augmented waveforms, as shown in Fig. 2. Transmitter and receiver positions may be entered as coordinates or placed by clicking on the rendered geometry, which ray-casts onto the scene surface. The experiment configuration records the Sionna version, maximum interaction depth, enabled propagation mechanisms, ray sample count, antenna pattern, array dimensions, and tap interval.

C. Digital-Twin Scene Import

The ray-tracing view accepts a Sionna/Mitsuba Extensible Markup Language (XML) scene together with its referenced mesh and texture assets. A scene may therefore be built in Blender or another compatible modeling workflow and loaded without changing the channel or learning components. Transmitter and receiver positions, antenna arrays, carrier frequency, and solver settings remain experiment parameters rather than properties fixed by the imported scene.

The toolbox distribution includes a ready-to-run UT Austin digital twin covering approximately 0.8 km by 0.6 km. It combines a ground plane with OpenStreetMap-derived campus buildings assigned Sionna radio materials based on International Telecommunication Union (ITU) models and provides an urban example for configuring transmitter–receiver locations and inspecting simulated paths. Users may instead load their own Sionna/Mitsuba scene and retain the same channel and learning workflow.

The HCRO digital twin used in the demonstration illustrates this external-scene path. The approximately 4 km by 2 km scene represents the site’s topography and elevation together with OpenStreetMap-derived buildings and ground surfaces assigned Sionna ITU materials. It is an NRDZ-relevant example rather than a required toolbox asset. For either scene, the resulting path data include complex coefficients, delays, and angles for the transmitter–receiver geometry selected by the operator. Fig. 2 shows the HCRO scene loaded in the ray-tracing view.

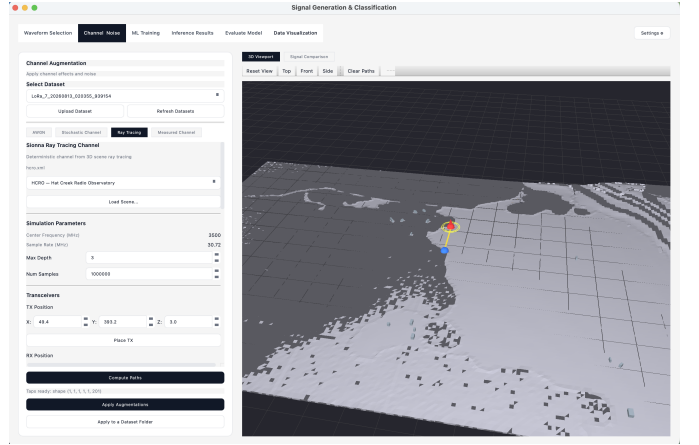


Fig. 2. Ray-tracing view with the imported HCRO scene, a placed transmitter–receiver pair, and the resulting propagation paths. The same view applies any bundled or user-supplied scene.

D. External Channel and Recording Import

The external-channel interface accepts a single file, directory tree, or archive containing user-supplied channel assets. MATLAB inputs may contain a one-dimensional complex impulse response or frequency response. NumPy inputs may use the same representation or an $N \times 2$ array whose columns store I and Q. The import configuration identifies the array variable, response domain, sample rate for an impulse response, frequency spacing or occupied bandwidth for a frequency response, center frequency when applicable, channel identifier, and normalization or calibration convention. Accepted arrays are converted to complex single precision, aligned with the waveform grid, and stored with their import metadata.

Recorded I/Q data may also be imported as a SigMF Recording comprising a `.sigmf-data` file and its `.sigmf-meta` description [6]. A supported complex datatype and `core:sample_rate` are required; capture frequency, class labels, and valid sample regions are read from capture and annotation metadata when available. To bound memory use and preserve local signal structure, each imported example is limited to the shorter of a configurable 10-ms window and 10^6 complex samples. Longer recordings are segmented into successive windows. Shorter regions retain their original length and are padded only when required by the selected model. SigMF recordings are treated as observed I/Q for training or evaluation, not as channel responses unless a channel estimate has been derived separately.

The CMAT collection used in the demonstration is one example of this general interface [4]. The training pipeline uses 308 `chanEst_*.mat` files, each storing a 1,200-bin complex frequency response under `chanEstCurrent`. A selected response is aligned with the generated waveform and applied to produce `augmented_data`. Let $X[k]$ denote the discrete Fourier transform of a clean waveform and let $H_m[k]$ denote imported channel m after frequency-grid alignment.

The augmented waveform is

$$y_m[n] = \text{IDFT}\{X[k]H_m[k]\} + w[n], \quad (2)$$

where $\text{IDFT}\{\cdot\}$ denotes the inverse discrete Fourier transform and $w[n]$ sets the requested post-channel SNR. The output retains the source filename, collection identifier, frequency-grid parameters, alignment operation, normalization choice, and random seed in its metadata. This provenance permits all examples derived from one measured channel to remain in the same data partition.

E. Dataset and Learning Components

Generated and augmented signals are stored as NumPy arrays with JavaScript Object Notation (JSON) metadata. The metadata record includes the signal class and generation parameters; augmented examples additionally record the selected augmentation and its configuration. The learning view discovers labeled examples, forms training and validation partitions, and trains one of four PyTorch architectures. The standard one-dimensional convolutional neural network (1-D CNN) contains 16- and 32-filter convolutional stages, while the compact 1-D CNN contains a single 8-filter stage; both use max pooling and adaptive average pooling before classification. These toolbox-specific baselines follow the general convolutional-network design of local receptive fields and shared weights [7]. The multilayer perceptron (MLP) contains hidden layers of 256 and 128 units and provides a feedforward baseline [8]. The one-dimensional residual network (1-D ResNet) uses an initial convolution followed by six two-convolution residual blocks at widths 64, 128, and 256 and global average pooling. It adapts residual skip connections to I/Q sequences rather than reproducing a published ResNet configuration [9]. Input width is selected from the data rather than the architecture: a complex baseband example is presented as two real channels holding I and Q, and a real passband example as one, so any of the four models can be trained on either representation.

Samples are padded or truncated to a fixed length before training, and each example is power-normalized so that absolute scale does not distinguish classes. A saved model carries a metadata sidcar recording its class labels, input width, signal length, hyperparameters, and the directories its training data came from, so two models that differ only in their channel condition remain distinguishable afterwards. The training view reports learning curves as the model converges (Fig. 3), and the evaluation view reports accuracy, per-class metrics, confusion matrices, classification reports, receiver-operating-characteristic curves, and low-dimensional dataset visualization (Fig. 4).

IV. LIVE DEMONSTRATION

The live demonstration shows how an operator uses the toolbox from end to end. It is organized as a two-stage walkthrough—data generation and training, followed by field-data evaluation and visualization—and is intended to make the integrated workflow concrete rather than serve as the paper’s primary contribution.

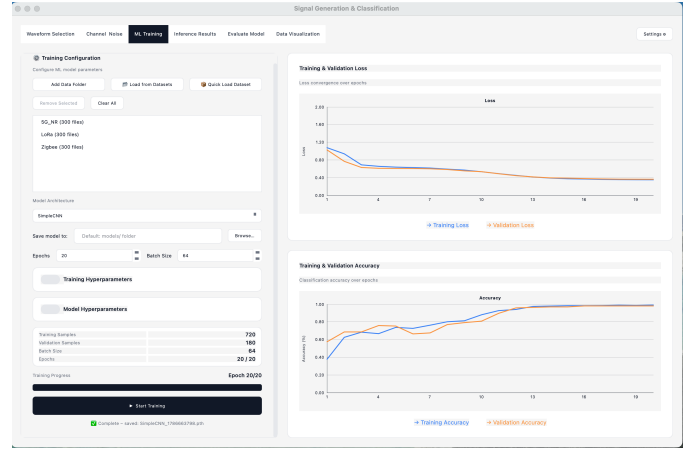


Fig. 3. Training view during a run, showing the loaded classes, the training configuration, and the learning curves.

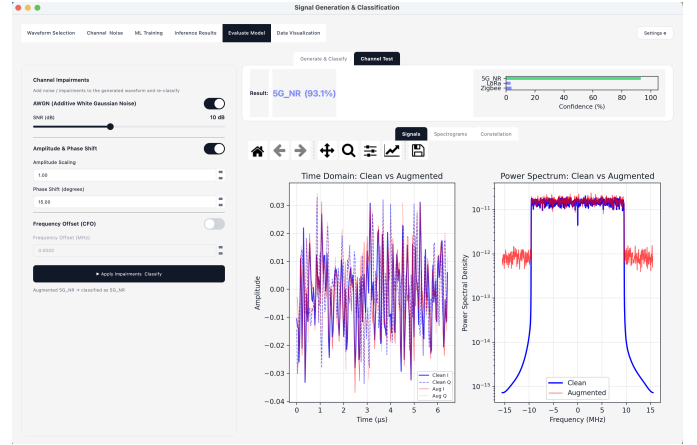


Fig. 4. Evaluation view reporting a confusion matrix and per-class metrics for a trained model scored on imported field recordings.

Stage 1—Data generation and training: The imported HCRO scene is loaded in the channel view, and a transmitter–receiver geometry is selected. The toolbox computes the propagation paths and channel taps and displays the scene together with the clean and channel-augmented signals. Labeled examples are generated in batches for the selected classes; clean copies and copies carried through the stochastic, ray-traced, and measured-channel paths form the training conditions. The training view applies the same compact PyTorch configuration to each condition and displays training and validation loss and accuracy as the models converge. The resulting models and class-label metadata are saved and transferred directly to the inference views.

Stage 2—On-stage evaluation and visualization: Known sequences are transmitted and recorded at stage with an Ettus USRP X410. The acquisition records the complex I/Q stream together with center frequency, sample rate, transmit and receive gains, timestamp, radio configuration, and valid labeled regions in SigMF metadata. The toolbox imports and presents the recordings through its graphical interface,

then applies the models from Stage 1 or other pretrained model of choice. Aside from accuracy, the inference view can additionally display per-class metrics, confusion matrix, and receiver-operating-characteristic curves. The same workflow can be repeated with different waveform families, channel sources, scenes, learning models, or field recordings. Fig. ?? summarizes the two demonstration stages. [TODO: HCRO campaign date and geometry; X410 center frequency, sample rate, waveform scaling, transmit/receive gains, capture duration and framing, synchronization procedure, and number of labeled windows.]

V. LIMITATIONS

Several limitations bound the present results. First, the standards-inspired waveform generators represent selected Wi-Fi, LTE, and 5G NR configurations rather than exhaustive implementations of those standards. Second, ray-tracing accuracy depends on scene geometry, radio-material properties, antenna models, and solver settings. Third, a static channel response does not reproduce mobility, long-term time variation, oscillator drift, nonlinear front-end effects, cochannel interference, or a complete receiver link budget. External responses require correct domain, rate or frequency-grid, and calibration metadata. SigMF imports represent recorded observations rather than channel responses unless an estimator is supplied, and windowing may omit behavior at longer time scales. Fourth, a ray-traced condition built from a single transmitter–receiver geometry supplies one channel realization for every generated example, whereas the stochastic and measured-bank paths draw a new channel per example; comparing them without accounting for this confounds propagation realism with channel diversity. Finally, the software supports classifier development and analysis but does not implement spectrum-policy decisions or automatic control of emitters.

Near-term work includes expanding measurement metadata, sweeping several ray-traced geometries so that path carries per-example channel diversity comparable to the stochastic and measured banks, validating simulated channels against site data, adding interference-composition experiments, and exploring out-of-distribution detection, finetuning, and continual learning on the platform.

VI. CONCLUSION

This paper presented a flexible, operator-facing RFML toolbox that connects parameterized waveform synthesis, channel augmentation, dataset provenance, model training, inference, evaluation, and visualization. Its primary contribution is a reusable end-to-end workflow in which researchers can exchange digital twins, stochastic models, measured channel responses, recorded I/Q data, and learning architectures without rebuilding the surrounding experiment. The bundled UT Austin scene and the imported HCRO and CMAT assets illustrate this flexibility without making any one site or dataset a dependency. The live demonstration shows how an operator moves from data generation and channel augmentation through training to evaluation of onsite recordings, providing evidence

of the flexibility of the workflow and of how it could be used in other projects. By reducing the integration work between signal, channel, learning, and analysis components while retaining experiment metadata, the toolbox is intended to support faster, more repeatable RFML studies for dynamic-spectrum and spectrum-coexistence research. The software is released at <https://github.com/ylyli/RFML-Toolkit>.

ACKNOWLEDGMENT

[TODO: Exact NSF award or subaward number, institution-approved funding statement and disclaimer, and acknowledgments for MITRE/SPARKIE, HCRO/SETI, scene and measurement contributors, and software and dataset providers.]

REFERENCES

- [1] T. J. O’Shea, T. Roy, and T. C. Clancy, “Over-the-air deep learning based radio signal classification,” *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 168–179, Feb. 2018, doi: 10.1109/JSTSP.2018.2797022.
- [2] U.S. National Science Foundation, “Spectrum Innovation Initiative: National Radio Dynamic Zones (SII-NRDZ),” NSF 22-579, 2022. [Online]. Available: <https://www.nsf.gov/funding/opportunities/sii-nrdz-spectrum-innovation-initiative-national-radio-dynamic-zones/505990/nsf22-579/solicitation>
- [3] MITRE, “MITRE-led team to develop radio dynamic zone for spectrum sharing research,” Nov. 19, 2024. [Online]. Available: <https://www.mitre.org/news-insights/news-release/mitre-led-team-develop-radio-dynamic-zone-spectrum-sharing-research>
- [4] S. Trudeau, S. Chun, B. Nguyen, and K. R. Chowdhury, “Measure once, train often: Scaling waveform classifiers with channel-augmented data,” in *Proc. IEEE/IFIP Int. Symp. Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, Columbus, OH, USA, 2026, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/document/11568203>
- [5] J. Hoydis, F. Ait Aoudia, S. Cammerer, M. Nimier-David, N. Binder, G. Marcus, and A. Keller, “Sionna RT: Differentiable ray tracing for radio propagation modeling,” in *Proc. IEEE GLOBECOM Workshops*, 2023, pp. 317–321, doi: 10.1109/GCWshps58843.2023.10465179.
- [6] The GNU Radio Foundation, Inc., “The Signal Metadata Format (SigMF),” version 1.2.6, 2025, doi: 10.5281/zenodo.1418396. [Online]. Available: <https://sigmf.org>
- [7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, Oct. 1986, doi: 10.1038/323533a0.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.